2 DI asılılığı kənardan verməkdir

Kafedə sifariş qəbul edən kod qəbz hazırlamalıdır. Əvvəlcə sifariş kodunun içində birbaşa qəbz hazırlayan obyekt yarada bilərik. Sabah qəbzi başqa formada hazırlamaq istəyəndə sifariş kodunu da dəyişməli olarıq.

DI bu bağlılığı azaltmaq üçün istifadə olunur. Sifariş kodu lazım olan qəbz hazırlayıcısını kənardan alır.

Dependency bir kod parçasının işləmək üçün ehtiyac duyduğu başqa obyektdir. Azərbaycan dilində “asılılıq” deyəcəyik. **Injection** həmin asılılığın kənardan verilməsidir. İkisi birlikdə **Dependency Injection**, qısa olaraq **DI** adlanır.

Məntiq

```
Sifaris kodunun isi: sifarisi qebul etmek.
Qebz kodunun isi: qebz hazirlamaq.

Sifaris koduna hazir qebz hazirlayicisi verilir.
```

İlk fikir budur: **bir sinif lazım olan hər obyekti özü yaratmağa məcbur deyil. Onu kənardan ala bilər.**

Bu fikri başa düşmək üçün hələ container və lifetime sözlərini bilməyə ehtiyac yoxdur.

Sinif obyekt və konstruktor eyni şey deyil

Sinif obyektin necə qurulacağını və nə edə biləcəyini yazdığımız tərifdir. **Obyekt** isə həmin tərifdən yaradılmış konkret nüsxədir.

C#

```
public class ReceiptPrinter
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

var printer = new ReceiptPrinter();
```

`ReceiptPrinter` sinifdir. `new ReceiptPrinter()` onun obyektini yaradır. `printer` həmin obyektə çatmaq üçün saxladığımız dəyişəndir. `var` dəyişənin tipinin sağdakı ifadədən müəyyən edilməsinə imkan verir.

`Print` bir metoddur, yəni çağırıla bilən əməliyyatdır. `string` nəticənin mətn olduğunu bildirir. `return` həmin nəticəni çağıran verir. `public` bu hissənin kənardan əlçatan olduğunu göstərir.

Konstruktor obyekt yaradılarkən onun ilkin vəziyyətini quran xüsusi metoddur. Növbəti səhifədə qəbz hazırlayıcısını məhz konstruktor vasitəsilə sifariş obyektinə verəcəyik.

Nüsxə sözünün ingiliscəsi **instance**-dir. “İki instance yarandı” deyiləndə iki ayrı obyekt yaradılması nəzərdə tutulur.

Asılılığı əvvəl içəridə sonra kənarda yaradaq

Əvvəl sifariş sinfi öz qəbz obyektini yaradır:

C#

```
public class OrderService
{
    private ReceiptPrinter printer = new ReceiptPrinter();

    public string Complete() => printer.Print();
}
```

Burada `OrderService` həm qəbz hazırlayıcısının konkret növünü seçir, həm də onu yaradır. `private` sahənin yalnız sinfin daxilində istifadə olunduğunu bildirir. => qısa metod yazılışıdır; bu nümunədə sağdakı nəticəni qaytarır.

İndi həmin obyekti kənardan verək:

C#

```
public class OrderService
{
    private ReceiptPrinter printer;

    public OrderService(ReceiptPrinter printer)
    {
        this.printer = printer;
    }

    public string Complete() => printer.Print();
}

var printer = new ReceiptPrinter();
var orders = new OrderService(printer);
```

`this.printer` obyektin içindəki sahədir; sağdakı `printer` konstruktora gələn parametrdir. Parametr çağırış zamanı verilən dəyərdir.

Bu artıq DI-dır. Qəbz hazırlayıcısı sifariş sinfinin içində yaradılmır. Onu konstruktorla veririk. Bunun üçün ayrıca kitabxana istifadə etmədik.

Interface dəyişə bilən hissəyə müqavilə verir

Əvvəlki nümunədə asılılığı kənardan verdik, amma sifariş sinfi hələ də konkret `ReceiptPrinter` tipini tanıyır. İndi qəbzin həm mətn, həm PDF variantı olduğunu düşün.

Sifariş koduna “qəbzi necə qurursan?” təfərrüatı lazım deyil. Ona qəbz hazırlamaq əməliyyatı lazımdır. **Interface** həmin əməliyyatı vəd edən müqavilədir.

C#

```
public interface IReceipt
{
    string Build();
}

public class TextReceipt : IReceipt
{
    public string Build() => "Metn qebzi";
}

public class PdfReceipt : IReceipt
{
    public string Build() => "PDF qebzi";
}
```

İki sinif də `Build` əməliyyatını təqdim edir. `: IReceipt` həmin müqavilənin yerinə yetirildiyini bildirir. Konkret həllə **implementation**, yəni implementasiya deyilir. Burada metodlar izah üçün mətn qaytarır; real PDF faylı yaratmır.

`OrderService` artıq konstruktorda `IReceipt` qəbul edə bilər. Ona `TextReceipt` də, `PdfReceipt` də vermək olar. Sifarişin əsas axını qəbz növünə görə dəyişməz.

DI interface olmadan da mümkündür. Interface-in burada əlavə etdiyi fayda **dəyişə bilən konkret həll üçün sabit müqavilə** yaratmasıdır.

Container bizim yerimizə obyektləri birləşdirir

Layihədə çoxlu sinif varsa hamısını əllə yaratmaq yorucu olur. .NET bunun üçün **DI container** təqdim edir. Container hansı tipin hansı obyektlə təmin olunacağını saxlayır və lazım olanda obyektləri birləşdirir.

C# qeydiyyat fraqmenti

```
builder.Services.AddScoped<IReceipt, TextReceipt>();
builder.Services.AddScoped<OrderService>();
```

Birinci sətir container-ə deyir: “IReceipt istənəndə TextReceipt ver.” İkinci sətir OrderService-i də container-in yarada biləcəyi xidmət kimi qeyd edir. `builder.Services` qeydiyyat siyahısıdır.

Servis burada proqramın istifadə etdiyi köməkçi obyektdir. Ayrı server və ya mikroservis mənasına gəlmir.

OrderService-in konstruktoru IReceipt istəyirsə, container əvvəl qəbz obyektini əldə edir, sonra onu OrderService-ə verir. Xidməti container-dən əldə etməyə **resolve** deyilir.

Qeydiyyat sətirinin işləməsi obyektin həmin anda mütləq yaradılması demək deyil. Obyekt çox vaxt ilk dəfə istənəndə yaranır.

IoC idarəetmənin başqa mexanizmə verilməsi barədə daha geniş anlayışdır. Bu nümunədə yaradılmanı container idarə edir. DI bunun asılılıqları verməyə aid üsuludur. SOLID dərsində isə DI ilə dizayn prinsipi olan DIP-in fərqini görəcəyik.

Transient scoped singleton fərqini iki sorğuda görək

Container-ə yalnız “hansı sinif?” demirik. “Nə vaxt yeni nüsxə yarat?” qaydasını da veririk. Bu qayda **lifetime**, yəni xidmətin paylaşılma ömrüdür.

Eyni xidmət birinci HTTP sorğusunda iki dəfə, ikinci sorğuda yenə iki dəfə istənilsin. Aşağıdakı hər hərf ayrı obyekti bildirir:

Lifetime	Birinci sorğu	İkinci sorğu
Transient	A, B	C, D
Scoped	A, A	B, B
Singleton	A, A	A, A

Transient: container-dən hər yeni istəmə zamanı yeni nüsxə alınır. Ancaq artıq aldığın obyektdə metod çağırmaq yeni resolve deyil.

Scoped: eyni scope daxilində eyni nüsxə alınır. **Scope** bir qrup xidmətin paylaşılma və təmizlənmə sərhədidir. ASP.NET Core adətən hər HTTP sorğusu üçün bir scope yaradır. “Hər istifadəçi üçün bir” və ya “hər thread üçün bir” demək deyil.

Singleton: eyni əsas container daxilində həmin qeydiyyatın bir nüsxəsi paylaşılır. Başqa server prosesinin ayrıca container-i və ayrıca singleton-u olur.

Seçimin əsası budur: **bu obyektin vəziyyətini kimlərlə və nə qədər müddət paylaşmaq istəyirəm?**

Uzunömürlü obyekt qısaömürlü obyekti saxlayanda

Kafedə bütün gün işləyən bir xidmət təsəvvür et. O, yalnız bir sifariş üçün nəzərdə tutulmuş iş dəftərini alıb bütün gün özündə saxlayır. Sonrakı sifarişlər də köhnə dəftərlə işləyir.

Singleton-un scoped obyekti konstruktorla alıb özündə saxlaması buna bənzəyir. **Captive dependency** qısaömürlü asılılığın uzunömürlü obyekt tərəfindən tutulmasıdır.

Problem

```
Singleton hizmeti: butun tetbiq boyu yasayir.
    |
    +-- bir sorqunun DbContext-ini sahesinde saxlayir.
```

DbContext EF Core-un baza ilə iş vahididir. O, yüklənmiş obyektləri və dəyişiklikləri izləyə bilər. Onu növbəti dərsdə açacağıq. Çox vaxt bir HTTP sorğusu üçün scoped qeydiyyatdan keçirilir. Eyni DbContext-i paralel işlərdə paylaşmaq təhlükəlidir.

İki əsas həll var. Xidmət həqiqətən sorğuya aiddirsə onu da scoped et. Xidmət uzunömürlü fon işidirsə, hər iş üçün ayrıca scope açıb işi həmin scope daxilində bitir.

`IServiceScopeFactory` ayrıca scope yaradan .NET xidmətidir. Açılmış scope-dan alınan DbContext-i iş bitəndən sonra singleton sahəsində saxlamırsan.

Əsas səbəb: qısaömürlü obyektin real istifadə müddəti nəzərdə tutulmuş sərhədi aşır.

Obyekti kim təmizləyir və DI nəyi həll etmir

Fayl və baza bağlantısı kimi resursları iş bitəndə buraxmaq lazımdır. **Dispose** həmin təmizləmə əməliyyatının adıdır. `IDisposable` bu əməliyyatı təqdim edən müqavilədir. `using` isə blokdan çıxanda Dispose çağırılmasını təşkil edir.

Container-in yaratdığı və təmizlənməsini idarə etdiyi scoped obyekt scope bitəndə təmizlənir. Singleton əsas container bitəndə təmizlənir. Container-dən aldığın eyni asılılığı hər istifadəçi ayrıca dispose etməməlidir.

Dispose obyektin yaddaşdan dərhal silinməsi deyil. **GC**, Garbage Collector, proqramın artıq çata bilmədiyi idarə olunan obyektlərin yaddaşını sonradan geri ala bilər. İdarə olunan burada yaddaşa .NET-in nəzarət etməsi deməkdir. Resursun buraxılması ilə yaddaşın geri alınması ayrı işlərdir.

DI yaradılma və əlaqələndirməni asanlaşdırır, amma aşağıdakı problemləri avtomatik həll etmir:

- **Dairəvi asılılıq:** A B-ni, B də A-nı tələb edir. Məsuliyyətləri yenidən bölmək lazımdır.
- **Gizli asılılıq:** biznes metodu container-dən özünə lazım olanı axtarır. Bu yanaşmaya Service Locator deyilir; constructor-dan real ehtiyaclar görünmür.
- **Paylaşılan vəziyyət:** singleton istifadəçilərin məlumatını ortaq sahədə saxlayırsa məlumat qarışa bilər.

Özünü yoxla

Bir sorğuda scoped xidmət iki yerdə istənilir. **Eyni obyekt alınır.** Həmin xidmət ikinci sorğuda istənilir. **Yeni scope üçün ayrı obyekt alınır.**